

HW03: Logical Agents and Propositional Inference

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
josh.manchester@uccs.edu

Abstract

This report presents implementations of knowledge-based agents using propositional logic and Horn clause inference. The work includes (Part A) propositional logic equivalences and model checking, (Part B) forward chaining inference for Horn clauses, (Part C) a Wumpus World reasoning agent that makes safe moves based on percepts, and (Part D) resolution-based theorem proving. All implementations follow SOFA (Single Responsibility, Open/Closed, Functional, Abstraction) software engineering principles for improved maintainability and extensibility. Testing demonstrates correct logical reasoning with inference completing in under 1ms for small knowledge bases and the Wumpus agent successfully navigating a 4×4 grid.

Introduction

Knowledge-based agents use explicit representations of knowledge to make decisions through logical inference [Russell and Norvig(2020), Atyabi(2025)]. This assignment explores several inference techniques for propositional logic, focusing on practical implementations that can reason about environments like the classic Wumpus World problem.

The Wumpus World (what is it?) is a grid-based environment where an agent must navigate safely by reasoning about pits and a monster (the Wumpus) using only local percepts like breezes (near pits) and stenches (near Wumpus). The agent builds a knowledge base from observations and uses logical inference to determine which neighboring cells are safe to visit.

This work implements four key components: (1) propositional logic evaluation with truth tables for verifying logical equivalences, (2) forward chaining inference for Horn clauses with O(n) time complexity, (3) a knowledge-based Wumpus agent that performs two-step reasoning, and (4) resolution-based theorem proving using conjunctive normal form (CNF). All implementations were refactored using SOFA principles to separate concerns between computation, presentation, and orchestration.

Part A: Propositional Logic

Logical Equivalences

Propositional logic equivalences form the foundation of logical reasoning. Two important equivalences are De Morgan's Law and Contraposition.

De Morgan's Law states that $\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$. This equivalence allows negations to be pushed inward through disjunctions. According to Russell & Norvig (Section 7.4.2) and Professor Atyabi's Lecture 8 (Part II, Slides 45-50) [Atyabi(2025)], this transformation is essential for converting formulas to conjunctive normal form (CNF) during automated reasoning.

Contraposition states that $(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$. This equivalence is particularly useful in backward reasoning, where we work from a goal back to known facts.

Implementation The implementation verifies these equivalences by enumerating all possible truth assignments (models) and checking that both sides evaluate to the same truth value in every model. For two propositional symbols P and Q, there are $2^2 = 4$ possible models to check.

The evaluation function handles implications by converting them to disjunctions using the equivalence $P \Rightarrow Q \equiv \neg P \vee Q$. To avoid symbol replacement bugs (what bugs?), the implementation uses placeholder tokens (`__IMPLIES__`, `__NOT__`, etc.) before performing symbol substitution, then converts these placeholders to Python boolean operators.

Results Table 1 shows the truth table for De Morgan's Law verification. All four rows show equivalence (both sides have identical truth values), confirming the law.

Table 1: De Morgan's Law Truth Table Verification

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$\neg P$	$\neg Q$	$(\neg P) \wedge (\neg Q)$	Equiv?
T	T	T	F	F	F	F	YES
T	F	T	F	F	T	F	YES
F	T	T	F	T	F	F	YES
F	F	F	T	T	T	T	YES

Similarly, Table 2 verifies contraposition across all four models.

Table 2: Contraposition Truth Table Verification

P	Q	$P \Rightarrow Q$	$\neg Q$	$\neg P$	$(\neg Q) \Rightarrow (\neg P)$	Equiv?
T	T	T	F	F	T	YES
T	F	F	T	F	F	YES
F	T	T	F	T	T	YES
F	F	T	T	T	T	YES

Model Checking

Model checking determines entailment by checking whether a query is true in all models where the knowledge base (KB) is satisfied. Formally, $KB \models \alpha$ if and only if in every model where KB is true, α is also true (Lecture 8, Part II, Slides 35-40) [Atyabi(2025)].

Algorithm The implementation uses the TT-ENTAILS algorithm from Russell & Norvig (Figure 7.10):

1. Enumerate all 2^n possible truth assignments for n propositional symbols
2. For each model:
 - Check if all KB sentences are true in that model
 - If KB is true but query is false, record as counterexample
3. Return true if no counterexamples found (query true in all KB models)

Test Case Knowledge Base:

$$KB = \{P \Rightarrow Q, Q \Rightarrow R, P\}$$

$$Query = R$$

For 3 symbols $\{P, Q, R\}$, there are $2^3 = 8$ possible models. The algorithm finds only 1 model where KB is satisfied: $\{P=\text{True}, Q=\text{True}, R=\text{True}\}$. In this model, the query R is also true, so $KB \models R$ is confirmed.

Performance Model checking completed in 0.000089 seconds for the 8-model enumeration. Time complexity is $O(2^n \cdot m)$ where n is the number of symbols and m is the KB size, making this approach feasible only for small problems ($n < 20$).

Part B: Horn Clause Inference

Horn Clauses

A Horn clause is a disjunction of literals with at most one positive literal. Horn clauses can be written as implications: $L_1 \wedge L_2 \wedge \dots \wedge L_k \Rightarrow L$ where all literals are positive. According to Russell & Norvig (Section 7.5.4) and Lecture 8 (Part III, Slides 60-75) [Russell and Norvig(2020), Atyabi(2025)], Horn clauses enable polynomial-time inference through forward or backward chaining.

Forward Chaining Algorithm

Forward chaining is a data-driven inference algorithm with $O(n)$ time complexity, where n is the KB size. The algorithm (Russell & Norvig Figure 7.15) maintains:

- **count**: Number of unsatisfied premises for each rule
- **inferred**: Boolean dict tracking which facts have been derived
- **agenda**: Queue of facts known to be true but not yet processed

Algorithm steps:

1. Initialize $\text{count}[r] = \text{number of premises in rule } r$
2. Add all known facts to agenda
3. While agenda is not empty:
 - (a) Pop fact p from agenda
 - (b) If p is the query, return true
 - (c) Mark p as inferred
 - (d) For each rule r where p appears in premises:
 - Decrement $\text{count}[r]$
 - If $\text{count}[r] = 0$, add r's conclusion to agenda
4. Return false if agenda exhausted without finding query

Implementation Details

The implementation uses Python's `collections.deque` for the agenda queue ($O(1)$ enqueue/dequeue operations) and dictionaries for constant-time lookups. The algorithm tracks which facts triggered which rules for generating inference traces.

Key optimization: The premise count mechanism ensures each rule is checked at most once (when the last premise is satisfied), achieving the $O(n)$ complexity bound.

Test Cases and Results

Generic Knowledge Base KB with rules showing chaining behavior:

$$\text{Facts} : \{A, B\}$$

$$\text{Rules} : \{A \wedge B \Rightarrow C, C \Rightarrow D, D \wedge E \Rightarrow F\}$$

Query: D

Forward chaining trace:

1. Process A: Rule 1 premise count goes from 2 to 1
2. Process B: Rule 1 fully satisfied, derive C
3. Process C: Rule 2 fully satisfied, derive D
4. D found: Query entailed in 4 iterations

Result: Query D is entailed. Execution time: 0.000025s.

Query F is NOT entailed because premise E is missing (Rule 3 never fires).

Wumpus World Fragment KB encoding Wumpus World rules (no breeze means no adjacent pits):

$$\text{Facts} : \{\text{not_}B_{1,1}, B_{2,1}, \text{not_}B_{1,2}\}$$

$$\text{Rules} : \{\text{not_}B_{1,1} \Rightarrow \text{not_}P_{2,1}, \text{not_}B_{1,1} \Rightarrow \text{not_}P_{1,2}, \dots\}$$

Query: $\text{not_}P_{1,2}$ (is there no pit at location (1,2)?)

Forward chaining derives $\text{not_}P_{1,2}$ in 2 iterations from $\text{not_}B_{1,1}$, confirming that location (1,2) is safe. This demonstrates how the Wumpus agent will use inference to identify safe cells.

Part C: Wumpus World Reasoning Agent

Environment Setup

The Wumpus World (Russell & Norvig Section 7.2, Lecture 8 Part I Slides 25-40) [Russell and Norvig(2020), Atiyabi(2025)] is a 4×4 grid containing:

- Pits at specific locations (e.g., (3,1))
- Wumpus monster at one location (e.g., (1,3))
- Agent starting at (1,1), known to be safe

The agent receives percepts at each location:

- **Breeze:** Felt in squares adjacent to a pit
- **Stench:** Smelled in squares adjacent to the Wumpus

The agent must infer which neighboring cells are safe based on these percepts using logical inference.

Agent Architecture

The agent maintains:

- **Knowledge Base:** Horn clauses encoding Wumpus World rules
- **Position:** Current (x, y) coordinates
- **Visited:** Set of cells already explored
- **Inference Engine:** Forward chaining for safety queries

Wumpus Rules (where does no breeze mean no adjacent pits?): For each cell (x,y) and each neighbor (nx, ny):

$$\begin{aligned} not_B_{x,y} &\Rightarrow not_P_{nx,ny} \quad (\text{no breeze} \rightarrow \text{no pit}) \\ not_S_{x,y} &\Rightarrow not_W_{nx,ny} \quad (\text{no stench} \rightarrow \text{no Wumpus}) \end{aligned}$$

Two-Step Reasoning Process

For each step, the agent:

Step 1: Sense

- Receive percepts (breeze, stench) at current location
- Add percept facts to KB: $B_{x,y}$ or $not_B_{x,y}$, $S_{x,y}$ or $not_S_{x,y}$

Step 2: Infer

- For each neighbor (nx, ny):
 - Query: $not_P_{nx,ny}$ (no pit?)
 - Query: $not_W_{nx,ny}$ (no Wumpus?)
 - If both true, mark neighbor as safe

Step 3: Decide

- Choose first unvisited safe neighbor
- If none available, stop (no safe moves)

Step 4: Act

- Move to chosen cell
- Mark as visited and safe

Execution Trace

Test world configuration:

- Grid: 4×4
- Pit: (3, 1)
- Wumpus: (1, 3)
- Start: (1, 1)

Step 1: Position (1,1)

Percepts: Breeze=NO, Stench=NO

KB additions:

- $not_B_{1,1}$ (no breeze at start)
- $not_S_{1,1}$ (no stench at start)

Inference finds 2 safe neighbors:

- (2,1): Inferred $not_P_{2,1}$ and $not_W_{2,1}$ from $not_B_{1,1}$ and $not_S_{1,1}$
- (1,2): Inferred $not_P_{1,2}$ and $not_W_{1,2}$ from same percepts

Decision: Move to (2,1) (first unvisited safe cell)

Step 2: Position (2,1)

Percepts: Breeze=YES, Stench=NO

KB additions:

- $B_{2,1}$ (breeze detected, pit nearby!)
- $not_S_{2,1}$ (no stench)

Inference finds 1 safe neighbor:

- (1,1): Already visited
- (3,1): UNSAFE (cannot infer $not_P_{3,1}$ due to breeze)
- (2,2): Inferred $not_W_{2,2}$ but CANNOT infer $not_P_{2,2}$ (unsafe)
- (1,2): Safe (from Step 1 inference)

Decision: Move to (1,2)

Final State:

- Visited: {(1,1), (2,1), (1,2)}
- Safe cells inferred: 4 cells
- Unknown: 12 cells

The agent successfully avoided the pit at (3,1) by recognizing the breeze at (2,1) and choosing the alternative safe path to (1,2).

Performance

Each inference step (forward chaining query) completes in <0.001 s. The agent's two-step execution took 0.003s total, demonstrating that propositional inference is efficient for small grid sizes.

Part D: Resolution-Based Inference

Resolution Algorithm

Resolution is a complete inference rule for propositional logic that uses proof by refutation. According to Russell & Norvig (Section 7.5.2) and Lecture 8 (Part III, Slides 95-110) [Russell and Norvig(2020), Atiyabi(2025)], to prove $KB \models \alpha$, we show that $KB \wedge \neg\alpha$ is unsatisfiable by deriving the empty clause.

Resolution Rule: If clause C_1 contains literal L and clause C_2 contains $\neg L$, then:

$$\text{resolve}(C_1, C_2) = (C_1 - \{L\}) \cup (C_2 - \{\neg L\}) \quad (1)$$

CNF Conversion

Resolution requires clauses in Conjunctive Normal Form (CNF): a conjunction of disjunctions. Conversion steps:

1. Eliminate implications: $P \Rightarrow Q$ becomes $\neg P \vee Q$
2. Move negations inward (De Morgan's laws)
3. Distribute $\vee$ over $\wedge$

Example: $P \Rightarrow Q$ converts to the single clause $[\neg P, Q]$.

Implementation

The implementation represents clauses as immutable data-classes:

```
1 @dataclass(frozen=True)
2 class Literal:
3     symbol: str
4     is_negated: bool
5
6 @dataclass(frozen=True)
7 class Clause:
8     literals: Tuple[Literal, ...]
```

This provides type safety (can't mix literals and clauses) and immutability (preventing accidental modification during resolution).

The resolution loop:

1. Convert KB to CNF
2. Negate query and add to clauses
3. Repeatedly resolve pairs of clauses
4. If empty clause derived: ENTAILED
5. If no new clauses: NOT ENTAILED

Test Cases

Entailed Query KB: $\{P \Rightarrow Q, Q \Rightarrow R, P\}$

Query: R

CNF clauses:

$[\neg P, Q]$

$[\neg Q, R]$

$[P]$

$[\neg R]$ (negated query)

Resolution trace:

1. Resolve $[P]$ and $[\neg P, Q] \Rightarrow [Q]$
2. Resolve $[Q]$ and $[\neg Q, R] \Rightarrow [R]$
3. Resolve $[R]$ and $[\neg R] \Rightarrow []$ (empty clause!)

Result: ENTAILED (contradiction found in 3 steps, 0.000112s)

Non-Entailed Query KB: $\{P \Rightarrow Q, Q\}$

Query: P

CNF clauses:

$[\neg P, Q]$

$[Q]$

$[\neg P]$ (negated query)

No new clauses can be derived from these three. The algorithm reaches saturation without deriving the empty clause.

Result: NOT ENTAILED (0.000043s)

Complexity Analysis

Resolution has exponential worst-case complexity: $O(2^n)$ clauses can be generated for n symbols. However, for small problems ($n < 10$), resolution completes quickly. The implementation includes a 100-iteration limit to prevent infinite loops on complex formulas.

SOFA Refactoring

All implementations follow SOFA software engineering principles:

Single Responsibility Principle (SRP)

Each class/function has one reason to change. Example refactoring:

Before: Model checking function mixed computation and printing

```
1 def model_check(kb, query, symbols, verbose=
2     True):
3     # MIXING: computation + I/O
4     for model in all_models(symbols):
5         if kb_satisfied(model):
6             if verbose:
7                 print(f"Model: {model}") #
8                 I/O
9             if not satisfies(query, model):
10                 counterexamples.append(model)
```

After: Separated into pure computation and presentation

```
1 # PURE: Computation only
2 def model_check_pure(kb, query, symbols):
3     """No I/O, returns immutable results."""
4     satisfying = []
5     counterex = []
6     for model in generate_all_models(symbols):
7         if model.satisfies_kb(kb):
8             satisfying.append(model)
9             if not evaluate(query, model):
10                 counterex.append(model)
11     return entailed, satisfying, counterex
12
13 # SEPARATE: Presentation
14 class ModelCheckPrinter:
15     @staticmethod
16     def print_results(kb, query, ...):
17         print("=== Model Checking ===")
18         # Only formatting, no logic
```

Benefits: Logic testable without I/O, presentation can change independently.

Open/Closed Principle (OCP)

Open for extension, closed for modification. Used strategy pattern for algorithms:

```
1 class HornInferenceStrategy(ABC):
2     @abstractmethod
3     def infer(self, kb, query):
4         pass
5
```

```

6 class ForwardChainingStrategy(
  HornInferenceStrategy):
7     def infer(self, kb, query):
8         # Forward chaining implementation
9
10 class BackwardChainingStrategy(
  HornInferenceStrategy):
11     def infer(self, kb, query):
12         # Backward chaining implementation
13
14 # Can add new strategies without modifying
  engine
15 engine = HornInferenceEngine(kb,
  ForwardChainingStrategy())

```

Benefits: Easy to add new algorithms (e.g., DPLL, Walk-SAT) without changing existing code.

Functional Programming (FP)

Immutable data structures and pure functions:

```

1 @dataclass(frozen=True) # Immutable!
2 class InferenceStep:
3     iteration: int
4     fact_processed: str
5     new_facts_derived: Tuple[str, ...] #
6     # Tuple, not list
7     rule_applications: Tuple[...]
8
9 # Pure function - no side effects
10 def resolve_clauses(c1: Clause, c2: Clause):
11     """Same inputs always produce same
12     output."""
13     for lit1 in c1.literals:
14         if lit1.negate() in c2.literals:
15             return Clause(...) # New
16                                     immutable clause
17
18 return None

```

Benefits: Easier testing (no mocking needed), parallelizable, no accidental mutations.

Abstraction

Hide implementation details behind interfaces:

```

1 class InferenceEngine(ABC):
2     """Abstract interface - implementation
3     hidden."""
4     @abstractmethod
5     def infer(self, query: str) ->
6         InferenceResult:
7         pass
8
9 # Client code doesn't know if it's using
10 # model checking, resolution, or forward
  chaining
11 engine = InferenceEngine = get_engine()
12 result = engine.infer("R") # Works for any
  engine

```

Benefits: Clients depend on abstractions not concrete implementations, easy to swap algorithms.

Table 3: SOFA Refactoring Metrics

Metric	Before	After
Pylint Score	9.37/10	9.11/10
Pure Functions	2	12
Immutable Types	0	7 frozen dataclasses
Max Function Complexity	15+	8
Tests Passing	6/6	6/6
Lines of Code	1400	2100 (+50%)

Refactoring Metrics

The refactoring increased code size by 50% but improved maintainability, testability, and extensibility. All original functionality preserved through backward-compatible facade functions.

Conclusion

This assignment implemented four inference techniques for propositional logic: truth table enumeration, model checking, forward chaining for Horn clauses, and resolution. The Wumpus World agent demonstrated practical application of logical reasoning, successfully navigating a grid by inferring safe cells from percepts.

Key findings:

- Forward chaining achieves $O(n)$ time complexity, completing in <1ms for small KBs
- Model checking is exponential ($O(2^n)$) but acceptable for $n < 20$ symbols
- Resolution-based theorem proving successfully proves entailment through refutation
- Wumpus agent makes correct safety inferences, avoiding pits and Wumpus
- SOFA refactoring improves code quality while preserving functionality

The implementations are modular and extensible. Future extensions could include:

- DPLL algorithm for SAT solving
- WalkSAT for approximate inference on large KBs
- First-order logic reasoning for Wumpus World
- Probabilistic reasoning for uncertain percepts

All code achieves 9.11/10 Pylint score with 6/6 tests passing, demonstrating both correctness and professional software engineering practices.

AI Use Disclosure

This assignment was completed with assistance from **Claude Code (Sonnet 4.5)**, version `claude-sonnet-4-5-20250929`.

AI assistance included:

- Understanding algorithm concepts from Russell & Norvig textbook and lecture slides
- Code implementation and debugging (propositional logic, Horn inference, Wumpus agent, resolution)

- SOFA refactoring guidance and implementation
- Test suite development and verification
- LaTeX formatting and report generation

All code was reviewed, understood, and tested by the student before submission. The AI did not complete the assignment autonomously – student understanding, testing, decision-making, and verification were central to the process. All algorithms follow Russell & Norvig specifications with proper citations.

References

- [Atyabi(2025)] Atyabi, A. 2025. Lecture 8: Logical Agents Part I-IV. CS 4820/5820 Course Lecture Slides. University of Colorado Colorado Springs, Fall 2025.
- [Russell and Norvig(2020)] Russell, S. J.; and Norvig, P. 2020. *Artificial Intelligence: A Modern Approach*. Hoboken, NJ: Pearson, 4th edition. ISBN 978-0134610993.